AI On-Call Engineer

Architecture and implementation handover — Checkpoints 1 (Eyes) and 2 (Brain)

Status: implementation plan • Architecture: 3 LLM agents + deterministic supervisor • Scope: read-only investigation and diagnosis for Day 1

1. Executive decision

Build one Python application with a deterministic incident workflow controller and three logically distinct LLM roles: Telemetry Investigator, Root Cause Analyst, and Recovery Planner. For Checkpoints 1–2, implement only the first two agents. Recovery Planner, human approval, and mutating Action API integration are deferred to Day 2. No agent per Cart, Checkout, Payment, or other application service.

Why: service boundaries are not incident boundaries. An alert in Checkout can originate in Payment, Cart, infrastructure, or a shared dependency. Agents are divided by investigative responsibility, not service name.

2. System architecture

Alert / CLI input → deterministic supervisor → Telemetry Investigator ↔️ Root Cause Analyst → evidence validation ➔ required JSON schema + readable report. Shared incident state stores tool calls, evidence, hypotheses, requested checks, and decisions. Read-only adapters access metrics, traces, logs, and container health. Later: Recovery Planner ➔ policy gate ➔ explicit human approval ➔ restricted Action API ➔ recovery verifier ➔ report.

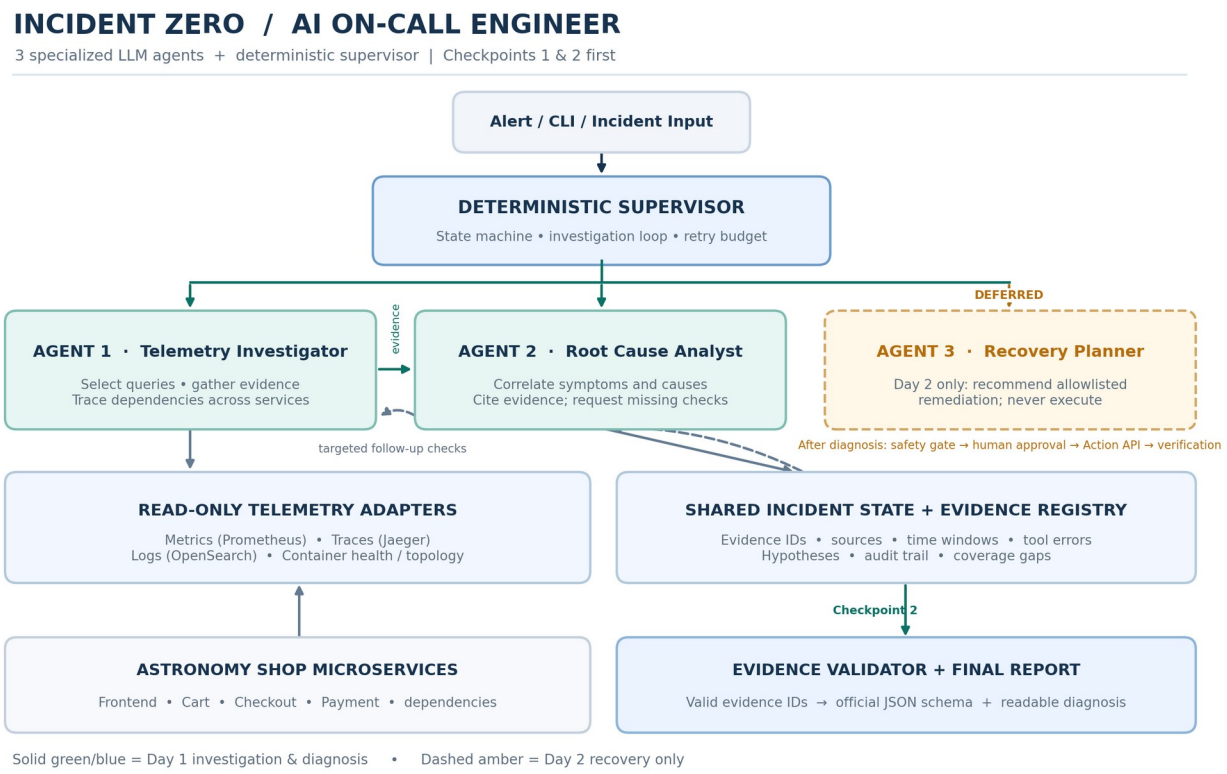


Figure 1. Checkpoint 1–2 investigation architecture; Recovery Planner and action path are deferred to Day 2.

Supervisor (non-LLM)

- Creates incident ID, deadline, and state; invokes workers and validates their structured outputs.
- Routes NEED_MORE_EVIDENCE back to Telemetry Investigator with targeted checks; routes DIAGNOSIS_READY to evidence validation and report; routes ESCALATE to human handoff.
- Enforces max investigation rounds, time budgets, read-only tool access, and no unsupported transitions.
- Does not make clinical-style probabilistic judgments or invent missing telemetry.

Agent 1 — Telemetry Investigator

- Accepts alert, service topology, existing evidence, and specific requested checks.
- Selects read-only queries for metrics, logs, traces, and container health; follows cross-service dependencies.
- Returns observations with timestamps, source, service, tool query, measured value/excerpt, and unique evidence IDs.
- Never declares a root cause or invokes mutating tools.

Agent 2 — Root Cause Analyst

- Accepts alert plus cited evidence; identifies symptom versus probable underlying fault.
- Maintains multiple hypotheses where appropriate, with supporting/contradicting evidence.
- Outputs NEED_MORE_EVIDENCE with targeted queries, DIAGNOSIS_READY with evidence IDs, or ESCALATE with reason.
- Must not invent metrics, trace IDs, logs, or certainty.

Agent 3 — Recovery Planner (Day 2)

Consumes validated diagnosis and the official action allowlist; proposes action, target, risk, preconditions, and verification. Cannot execute or approve changes. Do not build this before Brain passes.

3. Read-only tools and interfaces

- get_metrics(service, metric, start, end, filters): Prometheus or organizer-provided telemetry interface.
- get_traces(service, start, end, status, limit): Jaeger/OTel trace backend; preserve trace and span IDs.
- get_logs(service, start, end, query, limit): log backend; preserve log source and timestamps.
- get_container_health(service): read-only status, restarts, readiness and resource data where available.
- get_service_dependencies(service): documented service graph, optionally enriched from traces.
- All adapters enforce query limits/timeouts, return structured errors, and cannot mutate application state. Exact endpoints must be confirmed from starter kit.

4. Shared data contracts (internal; adapt to organizer schema)

Alert: incident_id, timestamp, affected_service, symptom, severity, window_start, window_end.

Evidence: evidence_id, tool, service, timestamp/window, query, observation, raw_reference, reliability_notes.

InvestigationResult: evidence[], investigated_services[], coverage_gaps[], tool_errors[].

DiagnosisResult: status = NEED_MORE_EVIDENCE | DIAGNOSIS_READY | ESCALATE; hypotheses[] = {service, cause, evidence_ids[], contradicting_evidence_ids[], confidence_label}; requested_checks[]; rationale; limitations.

IncidentState: alert, iteration, evidence_registry, requested_checks, diagnosis, audit_events, final_status.

Use Pydantic for internal contracts. Do not assume these internal fields match Bosch's final JSON schema. Load and validate against the supplied official schema for submission.

5. Supervisor state machine

RECEIVED → INVESTIGATING → ANALYZING. From ANALYZING: NEED_MORE_EVIDENCE → INVESTIGATING (within budget); DIAGNOSIS_READY → VALIDATING → REPORTING; ESCALATE or exhausted budget → ESCALATED → REPORTING. Validation failure can trigger targeted investigation or escalation. No mutating action is possible in Checkpoints 1–2.

Guardrails: maximum 3–5 investigation rounds (configurable), per-tool timeouts, total incident budget, bounded log/trace output, evidence deduplication, and explicit missing-data handling. Treat model confidence as a qualitative signal, not a calibrated probability.

6. Checkpoint 1 — Eyes (Day 1, 15:30 per detailed schedule)

Deliverables

- One-page architecture sketch showing supervisor, agents, read-only tool adapters, evidence store, and deferred action path.
- Working connection to the running Astronomy Shop and observability endpoints.
- Live evidence retrieval for at least one practice incident: metrics plus at least one of traces/logs, and container health where exposed.
- Saved raw tool output and a short demonstration showing the alert → query → evidence path.

Implementation sequence

- 0. Verify starter-kit instructions, compose/Kubernetes environment, service names, endpoint URLs, and credentials.
- 1. Implement read-only tool clients and test each independently with a fixed service/time window.
- 2. Normalize responses into Evidence objects with unique IDs and timestamped source references.
- 3. Implement a minimal CLI: python main.py investigate --alert <path>; print evidence summary.
- 4. Run one injected practice incident; show at least one real correlated signal and record any missing tools.
- 5. Export the architecture diagram and save a reproducible demo command.

Definition of done

The team can reproduce a real alert investigation on the laptop; the output cites real telemetry; read-only calls succeed or fail transparently; no Action API calls are made.

7. Checkpoint 2 — Brain (Day 1, 18:30 per detailed schedule)

Deliverables

- Telemetry Investigator and Root Cause Analyst integrated through the supervisor.
- Iterative NEED_MORE_EVIDENCE loop with targeted tool calls and bounded retries.
- At least three mentor-selected practice incidents diagnosed with real evidence.
- Final incident outputs validated against Bosch's official JSON schema, plus a human-readable explanation.

Implementation sequence

- 1. Implement Investigator prompt and structured InvestigationResult; prohibit unsupported claims.
- 2. Implement Analyst prompt and structured DiagnosisResult with evidence IDs and multiple hypotheses.
- 3. Add deterministic routing and evidence-reference validation; reject references absent from registry.
- 4. Implement schema-valid report exporter using the official organizer JSON schema.

- 5. Run three different practice faults; preserve alert, raw evidence, diagnosis, schema validation, and timing for each.
- 6. Test missing logs, failed tool calls, contradictory evidence, and exhausted investigation budget.

Definition of done

For three mentor-selected incidents, the system returns a grounded diagnosis or an explicit justified escalation, never a fabricated conclusion. All generated reports pass the supplied schema validator. Each major claim maps to evidence collected during that run.

8. Suggested repository layout

main.py; config.py; models/{alert,evidence,diagnosis,state}.py; agents/{telemetry,root_cause,recovery_planner}.py; tools/{metrics,traces,logs,container_health,dependencies}.py; workflow/{supervisor,validation}.py; reporting/{exporter,official_schema.json}; tests/{test_tools,test_routing,test_evidence,test_schema}; examples/; README.md. Recovery planner and action modules may remain placeholders until Day 2.

9. Team workstreams

- Workstream A — Observability: tool adapters, live endpoint connectivity, telemetry normalization.
- Workstream B — Agent reasoning: prompts, Pydantic contracts, evidence references, targeted follow-up queries.
- Workstream C — Integration and evaluation: supervisor, CLI, official JSON schema, three incident test runs, architecture sketch.
- Integrate early using shared contracts; avoid each member building a separate independent application.

10. Acceptance tests and failure modes

- Cross-service fault: checkout symptom, downstream payment/cart/infrastructure cause; investigator follows dependency graph.
- Multiple faults: analyst preserves more than one hypothesis rather than collapsing to first error.
- Missing telemetry: return coverage gap or escalation; no fabricated logs or metrics.
- Unsupported diagnosis: validator rejects nonexistent evidence IDs.
- Tool timeout: supervisor records error, retries within budget, then escalates.
- Schema mismatch: report rejected locally before checkpoint submission.
- No mutation: test that Day 1 agents have no write tool or action credentials.

11. Deferred Day 2 scope

Recovery Planner, deterministic action allowlist and safety policy, human approval tied to action parameters, kill switch, restricted Action API, verification and rollback/escalation workflow, complete audit trail. Keep these interfaces in the diagram but out of the Day 1 critical path.

12. Open items to confirm from starter kit

- Exact observability URLs, authentication, and supported queries.
- Official incident JSON schema and any required field names or output file conventions.
- Permitted models and networking restrictions.
- Actual action allowlist and approval mechanism (for Day 2).
- Checkpoint deadlines: the screenshots include differing schedule descriptions; confirm with mentors.

Handover principle: optimize for verifiable evidence, correct escalation, and repeatable demos—not number of agents.